Final Exam (Part 2)

Program Design and Data Structures (1DL201)

Teachers: Eva Darulova and Johannes Borgström

2024-03-14 / 8:00 - 12:00

Instructions

Read and follow these instructions carefully to increase your chance of getting good marks.

- This is a closed-book exam. You may use a standard English dictionary. Otherwise, **no notes, calculators, mobile phones, or other electronic devices are allowed**.
- Write your **exam code** at the top right corner of **each page**.
- There are **14 questions**. Each question awards a number of points, as listed in the question heading. You can obtain a maximum of **38 points**.
- Some questions ask for a justification of your answer; your justification needs to relate the information given in the question, the course content, and your answer.
- Some questions ask you to write code; this code needs to conform to the coding convention. You only need to write function documentation if the question text states so.
- Write your answers directly on the question sheet. You may use English and Swedish.
- Eva and/or Johannes will come to the exam hall around 9:00 to answer questions.

Question 1: Types (2 points)

Recall the list principle from course week 3: A list with elements of a given type is either null, or a pair whose head is of that type and whose tail is a list of that type.
How have we written this as a type in Typescript?
Answer:

```
type List<T> = null | Pair<T, List<T>>;
```

Question 2: Programming language processing (2 points)

For each of the following statements, state whether it is true or false.

	true	false
Both interpreters and compilers are programs.	☑	☐
Both interpreters and compilers execute programs.	☐	☑
An interpreter for TypeScript programs must be written in TypeScript itself.	☐	☑
Running a compiler on a program results in another program.	☑	☐

Question 3: Stacks (2 points)

Recall the stack data structure and its interface from the lectures:

```
export type NonEmptyStack<T> = [T, Stack<T>];
export type Stack<T> = null | [T, Stack<T>];
export function empty<T>(): Stack<T>
export function push<T>(e: T, stck: Stack<T>): NonEmptyStack<T>
export function top<T>(stck: NonEmptyStack<T>): T
export function pop<T>(stck: NonEmptyStack<T>): Stack<T>
```

and assume the following import statement in a file:

```
import { empty, push, pop, top } from '../lib/stack';
```

For the following code, state the result that is printed, or explain why the code results in an error and what the error is:
Answer:

```
const stck1 = push(33, push(4, push(18, empty())));
console.log(top(push(1, pop(stck1))));
```

1

```
const stck2 = push(24, pop(push(12, empty())));
console.log(top(pop(stck2)));
```

Type error; `top` expects a non-empty stack (type NonEmptyStack), but `pop` does not guarantee a non-empty stack as the return value (has return type Stack). The call to top is thus not type correct.

Question 4: Queues (2 points)

Recall the **mutable** implementation of the queue data structure. Order the following statements such that the contents of the queue at the end of the sequence is `[18, 1, 33]`:

```
enqueue(1, que1);        enqueue(33, que1);        enqueue(18, que1);

            enqueue(4, que1);           dequeue(que1);
```

You **must** use **all** of the statements.
Answer:

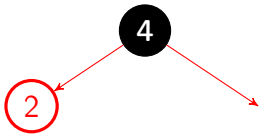
```
const que1 = q_empty<number>();
enqueue(4, que1);
enqueue(18, que1);
enqueue(1, que1);
dequeue(que1);
enqueue(33, que1);
display_queue(que1);
```

Question 5: Red-black trees (3 points)

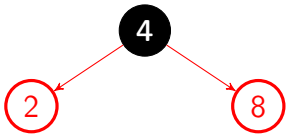
Insert elements with the keys 4, 2, 8, 5 and 6 into an initially empty red-black tree. We only care about the keys, and ignore the data associated with each node for simplicity.
Use the algorithm from the lecture and show the resulting tree after each (remaining) insertion. Circle red nodes, and leave black nodes without a border (do not use a red pen).
Answer:

insert 4:

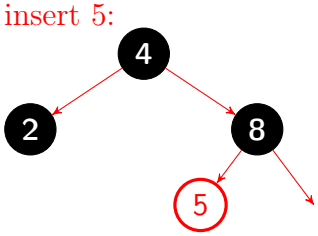
insert 2:



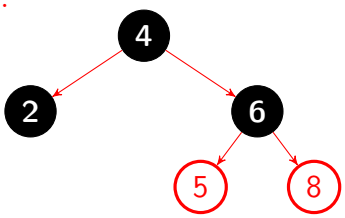
insert 8:



insert 5:



insert 6:



Question 6: Complexity (3 points)

What is the **worst-case time** complexity in terms of Big-Theta of the following operations?
Answer:

- push operation on a stack with n elements: $\Theta(1)$
- inserting an element into a heap with n elements: $\Theta(log\ n)$
- checking whether an edge between two nodes exists in a graph with v nodes and e edges that is given in the adjacency matrix representation: $\Theta(1)$

Question 7: Heaps (3 points)

Add a new element 6 to the following array heap using the algorithm seen in class (called MAX-HEAP-INSERT in I2A), showing the contents of the array at each intermediate step.
Answer:

9	5	7	4	3	6	2		(input array)
9	5	7	4	3	6	2	6	
9	5	7	6	3	6	2	4	
9	6	7	5	3	6	2	4	(array after last step)

Question 8: Records (3 points)

Recalling homework 5, the pixels in video frames in Source Academy were implemented using the following type, where the three first elements of the tuple were the red, green and blue intensities, respectively, and the fourth field (alpha) was unused:

```
type Pixel = [number, number, number, number];
```

Rewrite this type using a record type, using informative names for the fields. Also rewrite the following code that uses the Pixel type, so that it still performs the same transformation. Create a new Pixel record instead of using the function `set_rgba`.

```
// set_rgba(pixel, r, g, b, a) modifies pixel to be [r, g, b, a].

function color_rotate(src, dest) {
    for (let y = 0; y < image_height(); y = y + 1) {
        for (let x = 0; x < image_width(); x = x + 1) {
            set_rgba(dest[y][x],
                     blue_of(src[y][x]),
                     red_of(src[y][x]),
                     green_of(src[y][x]),
                     alpha_of(src[y][x]));
        }
    }
}
```

Answer:

```
type Pixel = { red:number, green: number, blue: number, alpha: number }

function color_rotate(src, dest) {
    for (let y = 0; y < image_height(); y = y + 1) {
        for (let x = 0; x < image_width(); x = x + 1) {
            dest[y][x] = { red:   src[y][x].blue,
                           green: src[y][x].red,
                           blue:  src[y][x].green,
                           alpha: src[y][x].alpha }
        }
    }
}
```

Question 9: Hashtables (3 points)

Assume that you have a hash table with 4 slots that uses quadratic probing to resolve collisions. Let $\perp$ denote an empty cell, and Δ a deleted one.
Can the hash function $h(s, i) = \texttt{length}(s) + i^2 \pmod 4$ produce the following table? Motivate your answer, showing every step.

0	1	2	3
PKD	$\perp$	IT	DVK

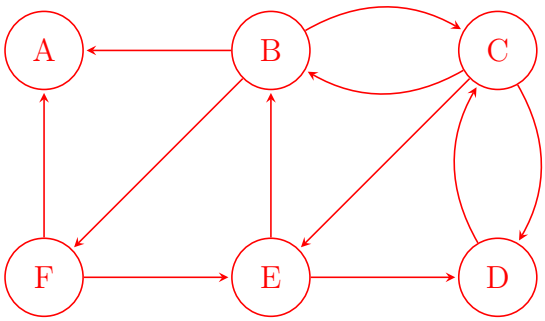
Answer and Motivation:
Yes: We start with an empty hash table.
First h("DVK", 0) = 3, so we insert "DVK" at 3.
Second h("PKD", 0) = 3, and we have a collision.
Then h("PKD", 1) = 0, so we insert "PKD" at 0.
Finally h("IT", 0) = 2, so we insert "IT" at 2.
(Note that we can insert "IT" first, second or last, yielding the same hash table.)

Question 10: Graph representation (2 points)

Consider the following directed graph, given in an adjacency list representation:

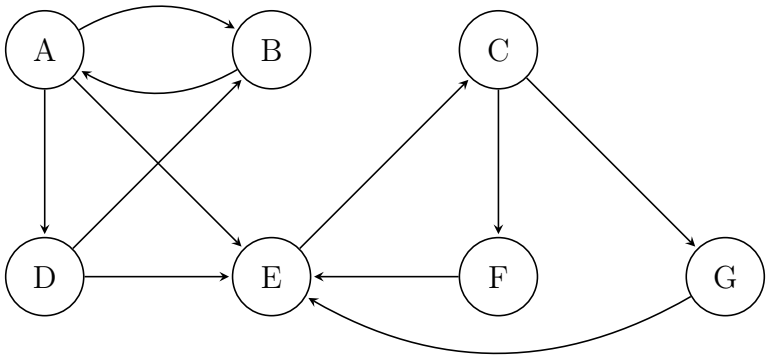
```
const graph = [list(), list(0, 2, 5), list(1, 3, 4),
               list(2), list(1, 3), list(0, 4)];
```

Draw the graph, with the nodes labelled A (with index 0) to F (index 5).
Answer:



Question 11: Graph traversal (3 points)

Consider the following graph.



Perform depth-first traversal of this graph, with restart order C, D, E, F, G, A, B. What are its discovery and finish orders (i.e., in which order are the nodes coloured grey and black, respectively)?

Answer:

C	F	E	G	D	B	A

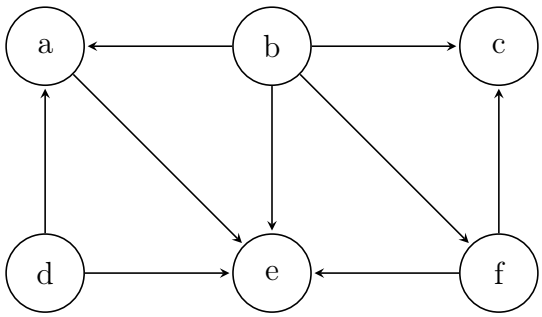
(discovery order, example)

E	F	G	C	A	B	D

(finish order, example)

Question 12: Topological sort (3 points)

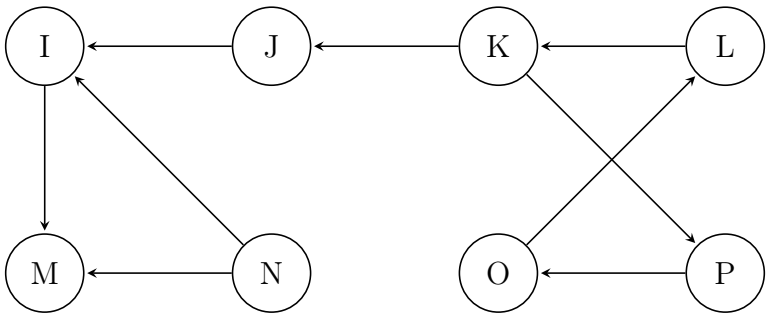
Consider the following graph:



Give one **valid** topological sort of the graph:
(for example) d - b - f - a - c - e
Give one **invalid** topological sort (i.e. an order that is **not** a topological sort) of the graph:
(for example) e - c - b - a - f - d
Explain your answer to the above two questions:
A topological sort is an ordering of all of the nodes in a graph such that if there is a path from node A to node B, then A appears before B in the ordering. This is the case for all of the nodes in the valid ordering. It is not the case for the invalid one, e.g. there is an edge from a to e, but e appears before a.

Question 13: Strongly connected components (4 points)

Consider the following graph:



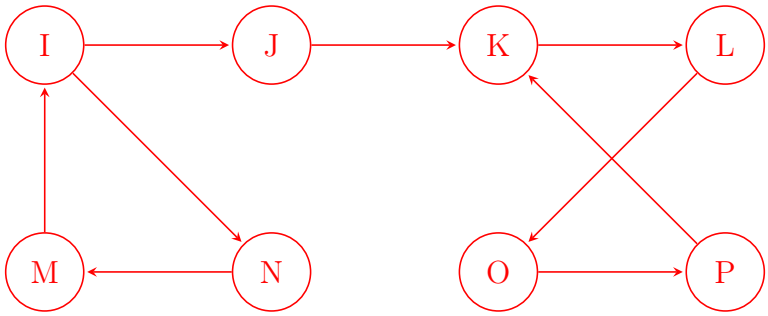
Use Kosaraju's algorithm (from the lecture) to compute the strongly connected components of this graph. Assume that the first step of the algorithm computed the following DFS finish order: `N, M, I, J, L, O, P, K`.

Steps of the algorithm:

[Correction at the exam: the edge between M and N was reversed; solutions that used the original graph were equally accepted.]

Step 1 of the algorithm is already computed (the DFS finish order).

Step 2 is to compute the transpose of the graph:



Step 3 is to do a DFS, using the reverse finish order as the restart order, i.e. `K, P, O, L, J, I, M, N`. Every time the DFS restarts, it completes a new SCC.

Starting from K, we discover the nodes K, L, O, and P. They form the first SCC. Then, restarting from J, we see that it is its own SCC, because K is already discovered. Finally, restarting from I, we discover N and M to build the third SCC.

Strongly connected components:

list(M, N, I), list(J), list(P, O, L, K)

[other solutions were accepted, depending on whether the correction was used or not]

Question 14: Meta-circular evaluator (3 points)

Recall the evaluation of function application in the MCE:

```
function evaluate(component: Component, env: Environment): Value {
    return is_application(component)
           ? apply(evaluate(function_expression(component), env),
                   list_of_values(arg_expressions(component), env))
           : ...
}
```

In which order are the different functions `f`, `g`, `h`, `j` and `k` called during the evaluation of the following expression?

`( f ( g() ) ) ( h(), j( k() ) )`

Answer and justification:

`g, f, h, k, j`

The top-level expression is a function application. Its function expression `f(g())` is evaluated first (line 3). To evaluate it, we first evaluate the function expression f (to f) and then evaluate the argument `g()`, evaluating `g` to itself and then *calling* `g` with an empty argument list. We then *call* `f` with the resulting value.
We evaluate the top-level argument expressions `h(), j(k())` from left to right (per the implementation of `list_of_values`). We first evaluate `h()` just as `g()` above, *calling* `h` with an empty argument list. To evaluate `j(k())`, we *call* `k()` before `j`, as in the evaluation of `f(g())` above.

This is the final page.